Práctica 5 -Transacciones JDBC (commit/rollback)

Venimos del CRUD con JDBC y de la reorganización por capas (Práctica 4).

Ahora damos el salto a casos de uso realistas, donde una acción toca varias tablas y no podemos permitir estados intermedios.

Una transacción es "todo o nada". O se guarda todo o no se guarda nada.

Una transacción es decirle a la BD:

"Voy a hacer varias cosas, y solo quiero guardarlas si todas salen bien". Por ejemplo, Si realizo una transferencia bancaria: o sale dinero de A y entra en B, o no quiero que pase nada. Si hay algún error intermedio necesito deshacer el proceso y volver al punto justo antes de comenzar la transacción.

Para ello, en JDBC debo tener en cuenta:

- .setAutoCommit(false) → empiezas "modo transacción" desactivando el auto-commit, ya que por defecto, JDBC trabaja con auto-commit true haciendo commit automático tras cada sentencia SQL.
- .commit() → confirma y guarda todo lo hecho desde que empezó la transacción (desde que pusiste autoCommit(false))
- .rollback() → deshaces todo desde el inicio de la transacción.

Nota importante: Si abres dos conexiones, ya no hay una única transacción. Por eso en un caso real hay que ejecutar TODO con la misma Connection. Para operaciones simples (un solo SELECT, un solo INSERT) puedes seguir con repos abriendo conexión internamente como hasta ahora, pero, en cuanto quieras transacciones (varias sentencias coordinadas) necesitas repos que acepten Connection.

En la práctica, lo que se hace es tener dos versiones de métodos:

- versión "normal" (abre su conexión) para uso simple
- versión "tx" para uso transaccional. El Service abre la Connection UNA vez y la pasa a todos los repos implicados como parámetro.

1) Objetivos

Partiendo del proyecto de la Práctica 4 (arquitectura por capas), implementa un caso de uso que haga un cambio en BD dentro de una transacción y fuerce un error a propósito para comprobar que el rollback revierte el cambio.

- Implementar try/catch con commit/rollback usando try-with-resources.
- Comprobar con una consulta SQL que el rollback ha funcionado.
- Aprender la regla: 'si quiero transacción, NO puedo abrir conexiones nuevas dentro de Repository'.

2) Qué modificamos

- TxLibroService (Service) con un método transaccional.
- TxDemo (App) para ejecutar y comprobar por consola.
- Pequeña ampliación en JdbcLibroRepository para ejecutar SQL con Connection compartida.

3) Qué se entrega

Capturas de pantalla con las salidas por consola y la tabla en pgAdmin para el caso en el que forzamos el rollback (Punto 5) y el caso en el que comentamos el fallo para ejecutar el commit (Punto 7).

4) Pasos a seguir

1. En pgAdmin, ejecuta: SELECT id, titulo, disponible FROM libro ORDER BY id; y elige un id existente (por ejemplo 1).
2. En IntelliJ, crea/revisa com.dam.ad.db.Db con getConnection().
3. En JdbcLibroRepository, añade métodos que acepten Connection (findByIdOrNull(Connection, id), update(Connection, libro)).
4. Crea TxLibroService en com.dam.ad.service (método marcarNoDisponiblePeroFallar).
5. Crea TxDemo en com.dam.ad.app y ejecútalo.
6. Vuelve a pgAdmin y repite el SELECT: disponible debe seguir igual.

Forzamos un error a propósito para comprobar que el rollback funciona. Es decir: el UPDATE se llega a ejecutar, pero NO queda guardado porque no se confirma con commit.

- ¿Qué significa “forzar el fallo”? Simulamos un error que podría ocurrir en la vida real (reglas, constraints, errores de red, etc.) con una línea: throw new RuntimeException("Fallo forzado…").
- ¿Se ejecuta commit en esta demo? NO. Al lanzar la excepción con throw, el flujo salta al catch y nunca llega a con.commit().
- ¿Entonces por qué el UPDATE no se queda en la base de datos? Porque con autoCommit=false los cambios quedan “pendientes” hasta commit. En el catch hacemos con.rollback() y se deshacen.

Flujo paso a paso (lo que ocurre cuando se ejecuta marcarNoDisponiblePeroFallar):

1) Se abre UNA Connection en el service y se llama a con.setAutoCommit(false). Empieza la transacción.

2) Se lee el libro con jdbcRepo.findByIdOrNull(con, id). Ojo: se usa la MISMA Connection.

3) Se hace jdbcRepo.update(con, l). El UPDATE se ejecuta contra la BD, pero todavía no está confirmado (no hay commit).

4) Se lanza el throw new RuntimeException(…). En ese instante Java deja de ejecutar el try y salta al catch.

5) En el catch se ejecuta con.rollback(): se deshace el UPDATE y cualquier cambio hecho desde el inicio de la transacción.

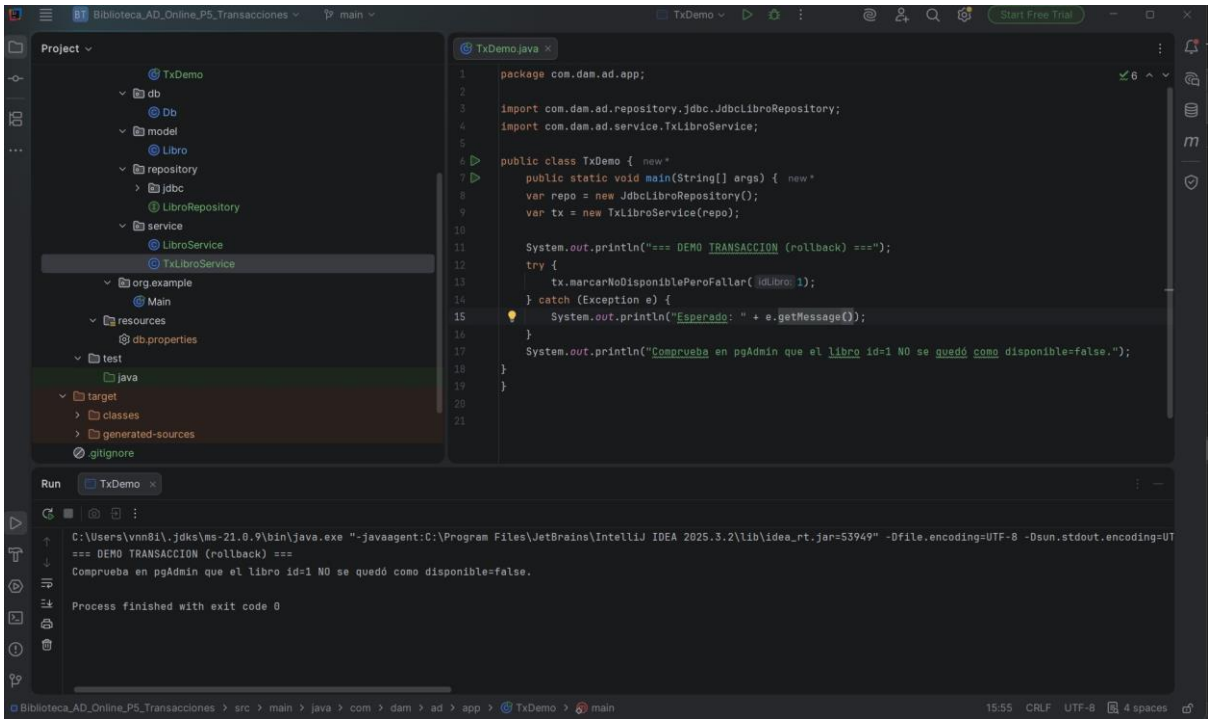
6) El commit no se ejecuta porque la excepción corta el flujo antes de llegar a esa línea.

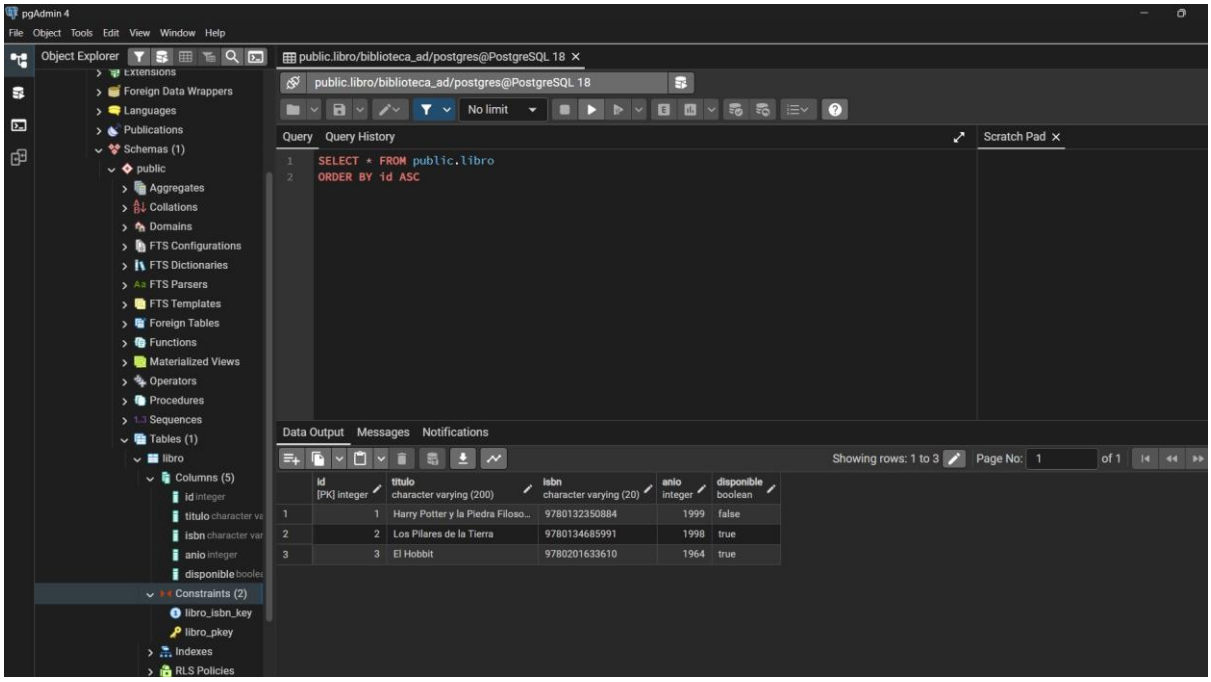
6) Problemas típicos rollback

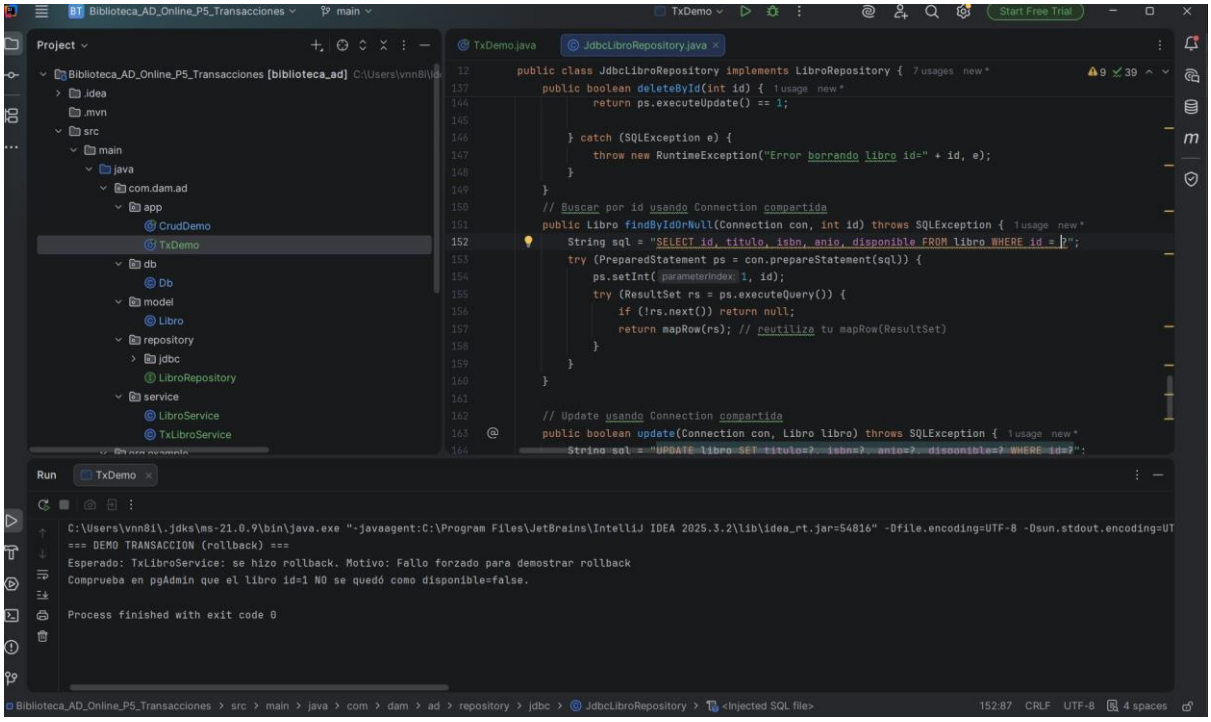
- “No hace rollback” → casi siempre: se olvido setAutoCommit(false).
- “Rollback pero el libro sí cambió” → están usando otra Connection (repo abre una nueva).
- “current transaction is aborted…” → hubo error y siguieron lanzando SQL sin rollback.

7) Demostrar el caso “éxito” (commit):

- Comenta la línea del throw (la del fallo forzado).
- Descomenta con.commit();
- Ejecuta de nuevo TxDemo y comprueba en pgAdmin que ahora SÍ queda disponible=false (porque hubo commit).





• Vuelve a dejar el throw activado para la entrega, ya que el objetivo de la práctica es ver el rollback.

Verónica Nicolás ACCESO A DATOS 2º DAM